
Teaching a 30B Model to Write Jac

CPT → SFT → DPO, and where the LoRA should live

Ayush · Jaseci Labs · August 2026

Qwen3-Coder-30B-A3B-Instruct · Apple Silicon · MLX

Agenda

Part 1 — the experiments

- What we are training, and how
- Does continual pretraining (CPT) help?
- Two engineering incidents worth the retelling
- Does smarter LoRA layer selection help?
- Five arms, one table — and the open question

Part 2 — the product

- Jac Model Studio (JMS): what is built
- Recent work and what is next

Close

- Takeaways and next steps

The setup

One model, one frozen dataset, one frozen holdout

Model

- Qwen3-Coder-30B-A3B-Instruct — a 30B mixture-of-experts model, $\sim 3\text{B}$ parameters active per token
- 4-bit quantized, $\sim 16\text{ GB}$ on disk; trained via LoRA adapters on a 48 GB Apple Silicon machine

Goal

- Generate **Jac** — a language with almost no presence in any pretraining corpus

Data (generated once, then frozen)

- 9,608 SFT rows / 826 DPO preference pairs
- Every arm trains on MD5-identical copies

Metric

- **Functional pass rate**: the generated code actually compiles *and* runs
- Same frozen **855-row** code-graded holdout, every arm, every stage

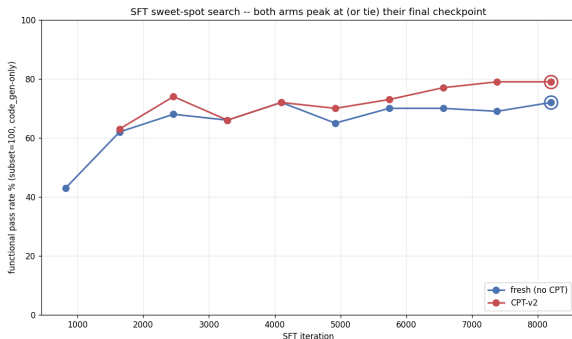
No LLM-as-judge, no BLEU. A row passes only if the Jac compiler and runtime agree.

The pipeline

Three stages, each answering a different question

- **CPT** — continual pretraining.
Keep reading raw Jac docs, papers, blogs. Teaches the *language*, not the task.
- **SFT** — supervised fine-tuning.
Show prompt/answer pairs. Teaches the *task*.
- **DPO** — direct preference optimization.
Show good-vs-bad pairs. Teaches *taste*.

SFT is the workhorse: it moves the model from single digits to ~70% on its own.



SFT sweet-spot search: neither arm would have gained from stopping early — the peak checkpoint ties the final one in both.

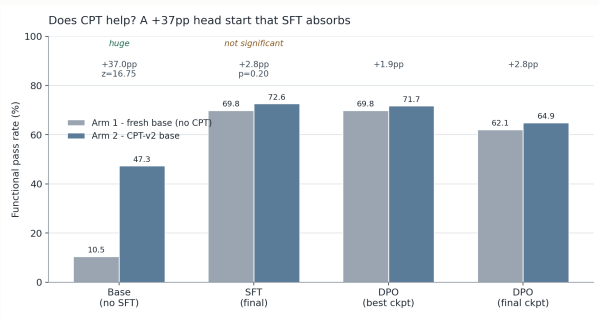
Two variables, five arms

Does CPT help? And which 16 of 48 blocks should carry the LoRA?

	Trailing-16 (blocks 32–47)	Spectrum picks (SNR-chosen)
No CPT (fresh base)	Arm 1	Arm 3
CPT-v2 (trained on trailing-16)	Arm 2 (incumbent)	Arm 4 (union/freeze)
CPT on Spectrum's picks	—	Arm 5 (paused)

- LoRA only adapts **16 of the model's 48 decoder blocks**. `mlx_lm`'s default takes the *last* 16 — a default nobody had ever measured.
- Everything else is byte-identical — rank 16, scale 2.0, dropout 0.05, 281.8M trainable parameters. **Placement changes; capacity does not.**

Question 1: does CPT help?



- Before any task training: **+37.0pp** — huge, unambiguous.
- After SFT: **+2.8pp**, $p \approx 0.20$ — **not significant** at $n=855$.
- The three deltas are **not the same kind of fact**. +37pp is a fact; +2.8pp is a direction.
- CPT-v2 had already been **rejected** on its own acceptance bar — 12 legs, all 16/16 on forgetting checks, but the semantic ceiling never moved.

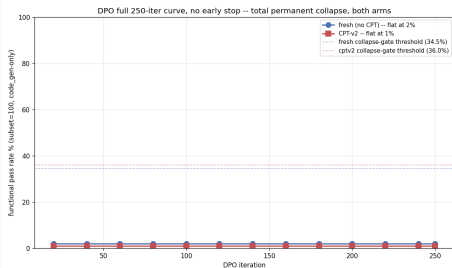
Two-proportion z -test throughout — this project's standing convention.

Incident 1: mlx_lm.fuse was quietly deleting the training

The symptom. Every DPO run “collapsed” to ~12% — immediate, total, identical across two arms, flat across all 13 checkpoints.

The tell. A failure that uniform is not an optimization problem. The model was not writing *bad* Jac — it was writing React.

Root cause. Fusing a LoRA delta into a 4-bit model dequantizes, adds, then *re-quantizes* — and re-quantization rounded the fine-grained SFT delta away. Every DPO run was training on an effectively un-SFT'd base.

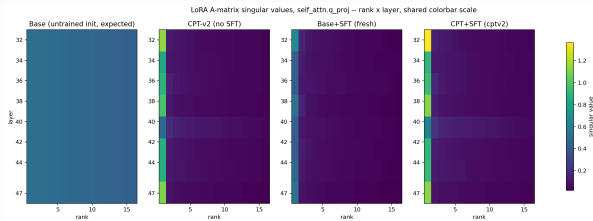


The “collapse” — entirely a measurement artifact.

Fix. Never fuse. Train DPO against the raw quantized model, seeded from the SFT adapter via `--resume-adapter-file`.

Corrected DPO ties SFT rather than destroying it.

Structural vs. functional: CPT's fingerprint survives



- If SFT erases CPT behaviourally, does it erase it *mechanically*? A singular-value probe on the `q_proj` LoRA updates says **no**.
- **CPT+SFT** tracks the standalone **CPT** adapter closely — same rank-1 dominance, same magnitude.
- **Fresh SFT** sits clearly apart: more spread, roughly half the rank-1 magnitude.
- Layer 47 stable rank: **2.70 / 2.83 / 4.42**.

Two differently-shaped updates reach the same pass rate. Convergence is behavioural, not mechanistic.

What is “Spectrum”?

Arcee AI’s signal-to-noise layer selection, in plain terms

- Take a weight matrix. Look at its singular values — how much “structure” it carries in each direction.
- Random matrix theory (Marchenko–Pastur) tells you exactly what that spectrum would look like if the matrix were **pure noise** of the same shape and scale.
- Anything *above* that noise floor cannot be explained by chance. It is learned structure.
- Rank every block by that ratio; train the top ones.

The intuition

Instead of guessing that the *last* 16 blocks matter most, measure which 16 actually carry signal.

Spectrum’s picks for this model:

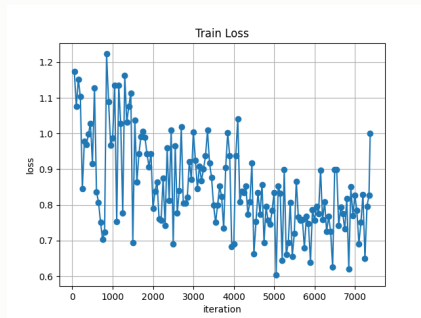
[0, 22, 23, 27, 30, 34, 36, 37, 38, 39, 41, 42, 43, 44, 45, 47]

Only **11 of 16** overlap the trailing slice — and it includes **block 0**, right next to the embeddings.

Question 2a: Spectrum on a fresh base — a clean win

Stage	Spectrum	Stock	p
SFT	74.7%	69.8%	0.023
DPO-best	74.2%	69.8%	0.046
DPO-final	72.7%	62.1%	<0.0001

- Significant at **every** stage, and the margin **grows** as training proceeds.
- Same capacity, same recipe, same data — only *where* the adapter sits changed.
- The trailing-16 default was leaving real capability on the table.



Arm 3 SFT training loss — 8,200 iterations, no incidents on this arm.

Question 2b: Spectrum on top of CPT — an honest null

Stage	Spectrum	Stock	p
SFT	70.5%	72.6%	0.34
DPO-best	71.7%	71.7%	exact tie
DPO-final	69.0%	64.9%	0.072

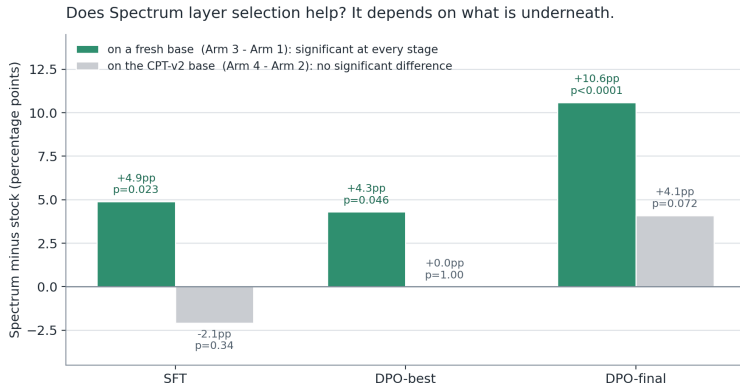
- No stage clears $p < 0.05$. One stage is even mildly negative.
- Reported as-is, not spun — an asymmetric result, not a clean win.

Why this arm is not a fair test

CPT-v2 had trained blocks 32–47. Spectrum wants a *different*, non-contiguous 16. Bolting one onto the other required converting the 21-block **union**, loading CPT-v2 into all of them, **freezing** the 5 CPT-only blocks, training 16, then merging the frozen 5 back into every artifact.

That union/freeze machinery is a **confound**, not just complexity — and it is exactly what Arm 5 exists to remove.

The asymmetry, in one picture



Spectrum is a real lever **when there is no prior adaptation to work around**.
On this evidence it is not a reliable lever once a CPT stage has already reshaped the same layer range.

Incident 2: a lazy memory-map silently zeroed a trained adapter

The symptom

- An 8,200-iteration run finished clean, exit 0, normal loss curve.
- Every trainable LoRA weight loaded back as **exactly zero**. No error, no warning.

The diagnostic that cracked it

- The **frozen** keys were fine; only the **trained** keys were zero.
- Frozen keys came from a *different* file. Trained keys came from the file the merge step overwrote in place.

Root cause

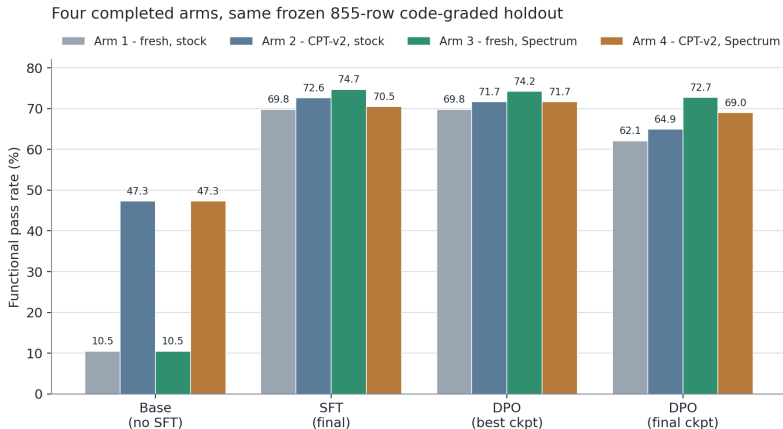
`mx.load()` on a safetensors file returns a **lazy memory map**. Writing back to the same path before those arrays are evaluated truncates the mmap they are still reading from.

The fix — one line:

```
trained = dict(mx.load(f))  
mx.eval(list(trained.values()))  
# ...then write
```

A generalizable MLX gotcha: **never read-then-overwrite a safetensors path without `mx.eval()` in between.**

Four completed arms, side by side



Best result to date: **Arm 3 — fresh base + Spectrum layer selection, 74.7% after SFT.**

All four arms scored on the identical frozen holdout, so every bar is directly comparable.

What is ahead: Arm 5

The one experiment that decides how to read Arm 4

The open question

- Was Arm 4's null caused by the **union/freeze confound** — CPT on one layer set, SFT/DPO on another?
- Or does Spectrum genuinely stop helping once *any* CPT is in the chain?

The design

- Re-run CPT itself on Spectrum's exact picks: one continuous 16-block lineage, base → CPT → SFT → DPO.
- No union, no freeze, **no merge step anywhere** — which removes the entire Incident-2 bug class.

Status: **paused, cleanly, resumable**

- Driver and downstream tree built; 5 pre-existing bugs found and fixed; 70 tests passing.
- All 3 pre-flight gates passed on the real 30B model — peak memory **30.1 GB**, ~6 GB under this machine's ceiling.
- Launched, ran leg 1, stopped intentionally before any checkpoint was written.

Honest budget: Spectrum includes block 0, so backprop traverses all 48 blocks — a measured $1.59\times$ slowdown. ~26–30 h for CPT alone, ~37–41 h end to end.

The other half: a local ML workbench, written entirely in Jac

- [illegible]

Jaseci Labs

JMS: recent work and what is next

Recent (July–August 2026)

- One design system replacing 5 dormant themes and a dead 7-theme switcher: light/dark, glass overlays, one accent colour.
- Workspaces group projects; the Experiments side seeds one per phase (SFT/DPO, RL, CPT, CPT-SFT).
- Assistant matured: streaming, persistent history, markdown tables, a local Gemma 3 4B slot loading in ~ 3 s.
- Accessibility pass: command palette, shortcuts, focus traps, toasts, job watch.

Next

- Confirm-reset hook on the irreversible cloud-terminate action; remaining ARIA hints.
- Exercise the untested paths: a real local model load, and the Claude-dependent success paths.

The live verification sweep (August 2026)

- ~ 20 scripted headless-browser sessions: 48 experiment screens, both themes, 6 overlays, a 234-action endurance run.
- **Zero console errors, zero uncaught page errors, zero 4xx/5xx.**
- Cold start **336–347 ms** to interactive across 8 fresh contexts.
- Focus traps held on 5 overlays, zero escapes.

Takeaways

- **SFT is the workhorse.** It takes this model from 10.5% to $\sim 70\text{--}75\%$. Everything else is a modest edit on top of that.
- **CPT's gift gets absorbed.** A +37pp head start on the raw model becomes a statistically invisible +2.8pp once SFT lands — though its *structural* fingerprint is still measurable in the weights.
- **Unexamined defaults cost real points.** Simply moving the LoRA to SNR-chosen blocks bought **+4.9pp** at zero extra capacity or cost.
- **But the same lever failed on a different base.** Reported as a null, not buried — and Arm 5 is designed to find out why.
- **Two silent-failure bugs, both in the measurement path, not the science.** A re-quantization that erased a delta, and a lazy mmap that zeroed a whole adapter. Neither raised an error. Both changed conclusions.
- **Design out the bug class, not the bug.** Arm 5 has no merge step at all — so Incident 2 cannot recur there.

Next steps

Experiments

1. Resume Arm 5's 12-leg CPT run (~26–30 h, resumable in chunks).
2. Run its SFT (~3.3 h) and DPO (~1.2 h) stages plus evals.
3. Publish the three-way report: six z -tests, Arm 5 vs Arms 2 and 4 at each stage — with the conclusions pre-committed before the numbers land.
4. Open follow-ups: more preference data for DPO (654 pairs is thin), and whether the SVD fingerprint holds beyond `q_proj`.

JMS

1. Close the remaining must-fix and should-fix items from the verification sweep.
2. Cover the deliberately-untested paths end to end.
3. Longer horizon: cloud dispatch and real log parsers for non-MLX backends; a dynamic section registry; per-user workspace isolation.

Part 2

NightShift

An autonomous overnight agent harness —
and the night it taught us what a ceiling isn't

A different project, a different failure mode.
Every figure is measured from the harness's own artifacts.

What NightShift is, and why it exists

The problem

- `jaseci-labs/jac`: ~**365,000 lines** of Jac, eleven packages, a compiler written in the language it compiles.
- It accrues the debt nobody is paid to fix: dead code, abstractions that were right two refactors ago, archetypes no test touches.
- Each item is small, local, mechanically verifiable, independently shippable — and nobody wants to supervise it turn by turn.

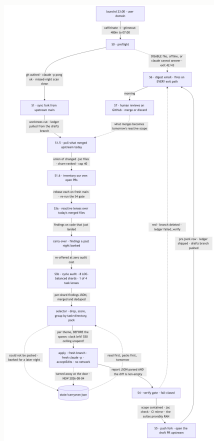
The design constraint behind everything downstream: it is unsupervised, so every stage has to be safe when it is wrong — and **honest about not having run**.

What it does

- Headless Claude Code sessions against the repo, **23:00–07:00**, unattended.
- Four rotating audit lenses → selection → one branch and one fresh agent per theme.
- Every branch runs a **local replica of upstream CI** — the CI a fork PR cannot reach. Not green, deleted.
- Survivors open as **draft** PRs; `ns_gh_write` refuses `pr ready` and `pr merge`. **A human merges, or nobody does.**

The pipeline, end to end

S0–S6 run nightly and unattended; S7 is the human loop



- **Entry.** launchd at 23:00, caffeinate, gtimeout 480m in lockstep with wallclock_min = 480 — the window enforced twice, by two mechanisms that cannot silently disagree.
- **S0 Preflight.** Lock, DISABLE kill file, every binary by absolute path, gh auth, and a literal pong probe proving Claude answers. Separates “launchd fired and the run died” from “launchd never fired”.
- **S1 Sync.** Fork synced from upstream main, worktrees cut, stale branches pruned, ledger pulled down.
- **S1.5 Merge poll (agent-free).** Union of the .jac files our merged PRs touched, churn-ranked, capped at 40. If the query *fails* it says so — it does not report an empty answer.
- **S1.6 PR inventory (agent-free, before any new work).** Rebaser our open PRs, re-run the S4 gate. **Existing PRs outrank fresh findings.**

There is deliberately no S2: the tier-1 autofix stage was retired 2026-07-30, and the gap is kept so older log lines still read true.

S3 → S7: audit, select, apply, gate, ship

S3 — the agentic tier, in strict priority order

- **S3a Reactive.** Lenses over the files that merged upstream *today*.
- **Carry-over.** Findings a past night paid to discover, re-offered at **zero audit cost**. 103 queued this morning.
- **S3b Cycle.** Tonight's *one* task — dead-code, abstraction, maintenance, coverage — over 8 LOC-balanced shards, read-only, \$8 cap. A session that *died* is a dead lens, never salvaged into “zero findings”.
- **Selection.** `selector.jac`, a pure unit-tested function: drop, score, group by **task + directory** — not the agent's free-text hint, which once made **105 groups from 112 findings**, 99 singletons.
- **Apply.** One theme, one branch, one fresh session: **no push, no gh, no network**.

S4 — Verify gate, fail-closed, cheap jobs first

- Base resolves → **scope containment** (anything outside the theme's files is treated as possible prompt injection) → `jac check baseline-diff` → CI-mirror fast jobs → the mirrored suites → **a positive assertion that the suites actually ran** → pre-commit → contribution rules.
- Anything red deletes the branch.

S5 — Ship

- Push on an explicit refspec, never forced; open upstream as a **draft**, asserting on rc 0 *and* a URL-shaped result — either alone can lie.

S6 — Digest S7 — Human loop

- Email built from the night's own artifacts, fired from an EXIT trap on *every* exit path.
- Merge, discard or leave. What stays open is re-gated nightly; what merges becomes tomorrow's reactive scope.

bash sequences processes, Jac owns every transformation, and there are **no Python files**.

The cost model

An unattended fleet of Opus sessions is, literally, a machine for spending money while you sleep

Layer 1 — per-session caps

- \$5 per apply, \$8 per shard, \$8 per reactive lens.
- Separate numbers for an arithmetic reason: reactive lenses cost **\$0.139/turn** against the shards' **\$0.0638/turn** — $2.2\times$. One shared cap would quietly move money from shards that produced **95 findings for \$38.36** to lenses that produced **5 for \$28.72**.

Layer 2 — the night ceiling

- `night_budget_usd = 50` — the only number that bounds spend, because **a per-session cap times fifty sessions is not a brake**.
- The measurement behind it: one night spent **\$76.55 in 97 minutes** of a 480-minute window — \$67.36 of it audits, only \$9.19 on the eleven applies that shipped all six branches.

Layer 3 — how spend is tracked

- `logs/<date>/spend.txt`: one `session_id / total_cost_usd` row per session, appended *at the session*, not reconstructed afterwards.

Both halves of that sentence are scar tissue

Totalling a night from the envelopes on disk gave **\$152.82 across “50 sessions”** — exactly double, because every meta- file is a byte-twin of the one it came from. Hence the `session_id` dedupe.

And an audit envelope is *overwritten in place* by its own retry, so two real attempt-1 Opus sessions survive in no file at all.

`ns_spend_check` sums that ledger and **fails closed**: a missing ledger is legitimately \$0.00, but a junk one stops the work.

The carry-over loop

The unit of memory is the *finding*: `fingerprint = sha1(file + rule)`

Why it is load-bearing, not a nicety

- On 2026-08-03 the night surfaced **182 findings** — 79 fresh, **103 carried** — packed **26 themes** and deferred **105 findings**.
- Audit money is spent *at discovery*. A finding discovered and then forgotten is money burned *twice*: the next audit rediscovers it and charges again.
- Carry-over is what converts audit spend into shipped PRs *across* nights instead of within one.

One governing constraint: RECONCILIATION B7

- Carry-over belongs to the *cycle* phase only. Reactive *outranks* carry-over; if it also absorbed it, yesterday's deferrals would be spent twice in one night and then overwritten. **B7 forbids displacement.**

The lifecycle

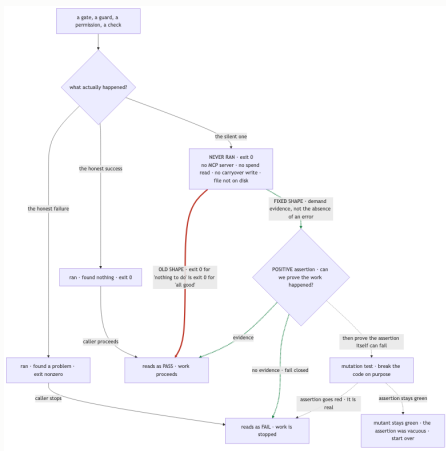
- `new` → `in_theme` — selected and applied. Recorded since 08-02, which stops the cycle phase re-buying what reactive already has a branch for.
- `new` → `deferred` — did not fit theme, night or clock at selection, **or** (since 08-04) was turned away at apply time. Same file, same schema either way.
- `deferred` → `in_theme` — re-packed later at zero audit cost. Carried findings sort first, so the carried copy wins the (file, rule) dedupe and keeps its flag.
- `in_theme` → `drafted` → `shipped`, or → `failed_verify` → `retry once` → `rejected`.
- `blocked` and `file_gone` are **terminal, and deliberately not carried.**

The one defect class this whole design is shaped around

The defect

“Did not run” scoring as “passed.”

- **Thirteen-plus instances** in this harness. Every one was in a gate or a guard.
- **None was caught by a failing test** — all were found by reading, or by an expensive night.
- The cause is always one shape: a command exits 0 for *nothing to do* and exits 0 for *all good*, and the caller cannot tell them apart.
- So is the countermeasure: **a positive assertion that the work happened**, not merely that it did not fail.



The catalogue — and the sibling class

A sample, because the range is what convinces

- ledger prunable wanted five argv where the caller passed four — so for four days every call printed usage, **exited 0**, and pruned nothing.
- selector's usage arm also exited 0, and its caller reads it on stdout. Arity drift would give an empty selection, a log line reading “no themes tonight”, and a night reporting *success* having shipped nothing. It exits **2** now.
- set -e does not fire on for x in \$(false), so a malformed shard table made the audit loop iterate **zero times** — no session ever started — and the night blamed the audit for a config error.
- upsert_theme hard-indexed a key coverage findings do not carry: a KeyError killed the **first live night** at S3. **\$17.42 spent, two finished branches thrown away**. Broken for four days, invisible because no coverage theme had ever survived an apply.

The countermeasure, everywhere

- assert_suite_ran / assert_check_ran, with a collection floor on the test counts.
- S1.6 separating “queried, zero PRs” from “the query failed”.
- S5 writing prs.jsonl *before* its early return, so an absent file proves S5 never ran.

The sibling class: assertions that cannot fail

- (set -e; ...) || fail is vacuous. A grep -q can match the comment instead of the code. A regression test can contain the bug it guards.
- So **every tripwire is paired with a mutation that must turn it red** — which caught one tripwire looking *past* the line it checked, and another whose mutant was written where jac run cannot load it, so “no bad output” was true for the wrong reason.

Fix 1 — a tool grant that granted nothing

2026-08-04, fix 1 of 4

Problem. Both allow-lists end with `mcp__jac__*` — the Jac language server's tools, how a session gets structural answers instead of grepping for them.

That server had never been registered against the live repo path.

Root cause. The registration is *local*, keyed by directory in `~/.claude.json` — not a `.mcp.json` in the repo. A move cut a fresh clone, and a fresh clone inherits nothing. **An allow-list entry naming a server that does not exist does not error; it simply matches no tool.**

Every session ran without the tools it was granted, and reported success: **the grant “did not run”, and scored as granted.**

Fix. `claude mcp add jac -- jac mcp` in the work tree — now the one piece of per-machine setup a fresh deploy needs, written into `WORKFLOW.md`. *The failure is silent, so the documentation is the tripwire.*

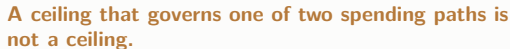
Second finding, unrelated blast radius

The global Claude Code settings carried a machine-wide `bypassPermissions` default. **NightShift's own sessions were never affected** — it names `--permission-mode` and an explicit `--allowedTools` on every invocation. But it was a standing hole for *every other* session on that machine. Removed.

2026-08-04, fix 2 of 4

Cycle brake fired	\$54.78 of \$50, at 00:19:30
Applies spawned after	16, from 00:26 to 01:56
Apply spend	\$60.25 / 26 — 55%
Audit spend	\$44.44 / 10 sessions

Root cause. The spend check ran only in the two audit fan-outs; the apply loop never consulted it — a *deliberate, documented and wrong* exemption. **Its premise assumed applies were 12% of spend. They were 55%.**



Verified against the *real* apply loop: a defect arm (ledger at \$99 → never spawns), a fails-closed arm (junk ledger), and **a positive control** (at \$1 it *must* spawn — else “no session started” passes vacuously).

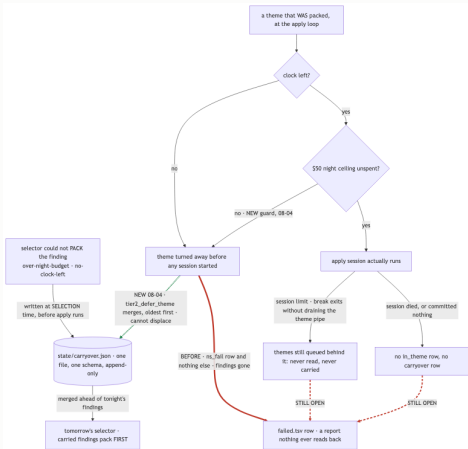
Fix 3 — carry-over remembered one way to fail, out of several

2026-08-04, fix 3 of 4

Problem. `carryover.json` only ever held findings the selector **could not pack in the first place**. A theme that was packed and was then turned away at the apply loop's door left **no memory at all** — no `in_theme` row (written only after a session succeeds), nothing in `carryover.json` (written *before* the apply loop runs), only an `ns_fail` row that *nothing ever reads back*. A report, not memory.

Harmless while the only apply-time guard was the clock, which fires at the *tail* of a night. **Fix 2 made it not harmless**: the cost ceiling defers themes from the *front* of the loop. Shipping the brake without this fix **would have turned a money leak into a silent work-deletion machine**.

Fix. `tier2_defer_theme`, called from **both** apply-time guards, projects the theme's findings back into the carry-over stream. **Same file, same schema** — a reader cannot tell an apply-time entry from a selection-time one — and **append-only**, so it merges rather than displaces, which keeps it inside B7.



Fix 3 — how it is proved, and what was left open

The strongest assertion of the day is a byte comparison

- One finding through the *real* selector twice: packed then turned away at apply, versus deferred by the selector itself. **The two outputs must be `cmp -s` identical.**
- A parallel mechanism that merely looks similar — different keys, a missing flag, a lost fingerprint — cannot pass that.
- **It appends, it does not replace:** yesterday's carry-over is seeded first and must survive. And **tomorrow can use it:** the file is fed back through the selector and must come out packed.
- **The false-positive arm**, the one that matters: a theme that *reached* its session must **not** appear — otherwise every finding the night shipped is re-bought tomorrow.

Two holes of the same shape — known, and explicitly NOT fixed

1. The **session-limit** break exits the apply loop without draining the theme pipe, so the themes behind it are never *read*.
2. A theme whose **session dies or commits nothing** still gets only an `ns_fail` row.

Both are real, and both were left alone rather than bundled into a fix already touching two guards — **named in the code, in `WORKFLOW.md`, and in the diagram itself.**

Standing rule: a known hole gets written down where the next person will trip over it, not filed elsewhere.

2026-08-04, fix 4 of 4 — and not the path it was given

Root cause. `drop_reason()` *had* an existence check — inside `impl_siblings()`, which checks the candidates **it derives itself**, never the finding's own file. **A derived guess and a stale input are the same failure with different provenance.**

And it goes stale *by design*: the audit reads a clone, the clone is refreshed, upstream renames in between. On the 07-31 replay a finding still packed a theme naming `planner.jac` after upstream renamed it `query_planner.jac`.

Fix. It now checks the finding's own file against the tree, returning a new reason: `file-gone` — **terminal and not carried**, since the next audit re-finds the issue at its new path, or not at all. And it **stands down when there is no tree to check**: emptying the night over a missing clone is the worse failure.



Verification. A Jac unit test on the real 07-31 shape, three arms: the stale finding drops as exactly file-gone, with **no** carry-over entry; **the false-positive arm** — the finding that *is* on disk still packs, because a rule that drops everything satisfies arm 1 perfectly and costs the harness its entire output; and **the stand-down arm**, proving a missing clone cannot silently empty the night.

What is still open

The budget sizing decision — deferred, not forgotten

- `night_budget_usd` is still **50**. Fix 2 makes the number *mean* something; it does not claim it is right.
- **Raise the cap** — 08-03 produced real work at \$109.73; if that is worth it, the ceiling should say so.
- **Cut audit spend** — \$44.44 bought 79 fresh findings against a queue already holding 103. **The bottleneck is not discovery.**
- **Accept audit-only nights** — banking everything for tomorrow. A *decision*, not a discovery.
- The data exists: cost, model, phase and findings are logged per session, so “findings per dollar” is a query.

The two remaining carry-over holes — the session-limit break, and the theme whose session dies or commits nothing.

A reporting gap that follows from fix 3.

`themes_deferred` counts *selection-time* deferrals only, so on a night whose apply loop hits the ceiling it **undercounts the real backlog**.

Readiness, honestly. The system is armed and unattended, and *young*: the v2 cutover was 08-02, and the first live night after it died at S3 on a `KeyError`. **Two of the last four nights found a defect in the harness rather than in the target repo** — a ratio expected to fall, and **measured**, not asserted.

Where the system stands tonight

- `bin/test-harness.sh`: 41 of 41 sections green — up from red at section 32 this morning.
- Every fix landed today was mutation-tested — each new assertion was verified to go *red* against the broken behaviour, not merely green against the fix. That is why they are believable at all.
- The night ceiling is now a ceiling. Both spending paths consult the same check against the same ledger, and both fail closed.
- The carry-over queue holds 103 findings, all already paid for, now fed by both deferral paths through one file, one schema, one reader.
- The jac MCP server is registered and connected, so the tool grants resolve to real tools; the machine-wide `bypassPermissions` default is gone.
- Nights are armed: loaded in `launchd`, no `DISABLE` file, window 23:00–07:00 with a 480-minute ceiling.

The through-line, one sentence

Every defect closed today was the same defect — something that did not run, and read as something that passed. The only reliable defence found here is to **demand positive evidence that work happened, then break the code on purpose.**

Thank you

Questions?

Ayush · Jaseci Labs · August 2026